

ASSIGNMENT Unit – 1 Introduction to Data Structures

Subject: Data Structure

Course: BCA – Semester 2 (GTU)

Q1. Explain Data Management Concepts in detail.

Answer:

Data Management refers to the process of collecting, storing, organizing, maintaining, and using data efficiently and securely.

Key Data Management Concepts:

1. **Data**
 - Raw facts and figures such as numbers, text, images.
 - Example: Student marks, roll number.
2. **Information**
 - Processed data that has meaning.
 - Example: Average marks of a student.
3. **Data Processing**
 - Conversion of raw data into useful information.
 - Includes sorting, searching, updating.
4. **Data Storage**
 - Saving data in memory or secondary storage.
 - Example: Files, databases.
5. **Data Integrity**
 - Ensures accuracy and consistency of data.
 - Example: Marks cannot be negative.
6. **Data Security**
 - Protecting data from unauthorized access.
 - Example: Password-protected systems.
7. **Data Organization**
 - Arranging data using proper data structures.
 - Example: Using arrays, linked lists.

Conclusion:

Proper data management improves performance, reduces errors, and ensures efficient data usage in computer systems.

Q2. Explain Data Types. Classify them into Primitive and Non-Primitive.

Answer:

Introduction

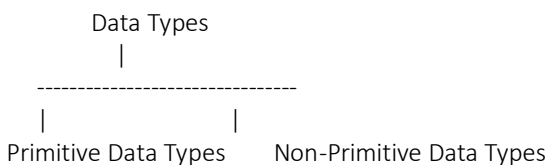
In computer programming, data is stored in memory using **data types**. Data type specifies **what kind of data** a variable can store and **how much memory** it requires. Proper use of data types helps in efficient data handling and program execution.

Definition of Data Type

A **data type** is a classification that defines the **type of value** a variable can store, such as numbers, characters, or logical values.

Classification of Data Types

Diagram: Classification of Data Types



1. Primitive Data Types

Definition

Primitive data types are **basic data types** that store **single values** and are directly supported by the programming language.

They require **fixed memory** and are simple to use.

Types of Primitive Data Types

(a) Integer (int)

- Stores whole numbers
- No decimal values

Example:

```
int age = 20;
```

(b) Floating Point (float / double)

- Stores decimal numbers

Example:

```
float percentage = 75.5;
```

(c) Character (char)

- Stores single character

Example:

```
char grade = 'A';
```

(d) Boolean (bool)

- Stores true or false values

Example:

```
bool isPassed = true;
```

2. Non-Primitive Data Types

Definition

Non-primitive data types are **complex data types** that store **multiple values** or **collection of data**. They are also called **reference data types**.

Types of Non-Primitive Data Types

(a) Array

- Stores multiple values of **same data type**
- Uses index to access elements

Example:

```
int marks[5] = {70, 80, 75, 90, 85};
```

Diagram:

```
Index: 0  1  2  3  4
-----
Array | 70|80|75|90|85|
```

(b) Structure

- Stores values of **different data types**
- Used to represent records

Example:

Student record (roll no, name, marks)

(c) Union

- Similar to structure
 - Shares memory among members
-

(d) String

- Collection of characters

Example:

```
char name[] = "Payal";
```

(e) Linked List

- Collection of nodes connected using pointers

Diagram:

[Data | Next] → [Data | Next] → NULL

Difference Between Primitive and Non-Primitive Data Types

Primitive	Non-Primitive
Stores single value	Stores multiple values
Simple data types	Complex data types
Fixed memory	Dynamic memory
Fast access	Slower access

Q3. Explain Performance Analysis and Measurement of Algorithms.

Answer:

Introduction

When a problem can be solved using different algorithms, we must choose the **best and fastest** one.

Performance analysis and measurement of algorithms help us understand how much time and memory an algorithm uses.

Importance of Performance Analysis

- Helps select best algorithm
- Saves time and memory
- Improves program efficiency
- Useful for large data problems

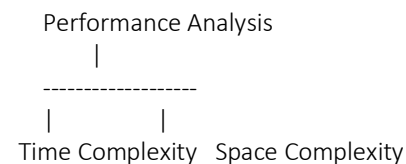
Performance Analysis of Algorithm

Meaning

Performance analysis means **studying how efficient an algorithm is** without actually running it on a computer.

It is done **theoretically**, before writing the program.

Performance analysis mainly depends on:



1. Time Complexity

Definition

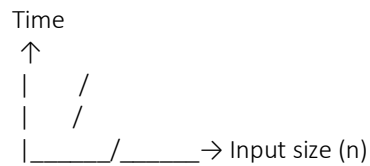
Time complexity shows **how much time an algorithm takes** to run when the **input size (n)** increases.

It counts **number of steps or operations**, not actual seconds.

Example

- Searching a number in a list of 5 elements takes less time
- Searching in a list of 100 elements takes more time

Simple Diagram



2. Space Complexity

Definition

Space complexity shows **how much memory an algorithm needs** to execute.

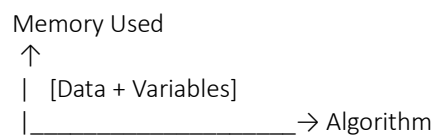
It includes:

- Input data
- Variables
- Extra memory used by algorithm

Example

- Algorithm using extra arrays needs more memory
- Recursive function needs stack memory

Simple Diagram



Measurement of Algorithm

Meaning

Measurement means **checking actual performance** of an algorithm by **running it on a computer**.

It gives **real execution time** and **memory usage**.

Types of Measurement

1. Time Measurement

- Actual time taken by program
- Depends on system speed

2. Memory Measurement

- Actual memory used during execution
- Depends on system configuration

Difference Between Analysis and Measurement

Performance Analysis	Performance Measurement
Theoretical	Practical
Done before coding	Done after coding
Hardware independent	Hardware dependent

Conclusion

Performance analysis and measurement help us understand the efficiency of an algorithm. Analysis gives an idea before implementation, while measurement confirms performance after execution. Both are important in algorithm design.

What is Data Structure? Explain various types of Data Structure.

OR

Discuss the various types of data structures with classification into linear and nonlinear types.

Introduction

In computer applications, a large amount of data is used daily. To store, organize, and process this data efficiently, data structures are used. Proper data structure selection improves program performance and reduces complexity.

Definition of Data Structure

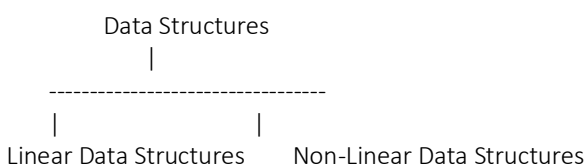
A **data structure** is a **systematic way of organizing and storing data in computer memory** so that data can be **accessed, modified, and processed efficiently**.

Example:

Storing marks of students using an array.

Classification of Data Structures

Diagram: Classification of Data Structures



1. Linear Data Structures

Linear data structures store data elements in a **sequential (linear) manner**, where each element is connected to the next one.

Diagram: Linear Data Structure

[10] → [20] → [30] → [40] → [50]

a) Array

Definition:

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations** and accessed using an index.

Diagram:

```
Index:  0  1  2  3  4
-----
Array: |10 |20 |30 |40 |50 |
-----
```

Example:

Storing marks of students.

b) Linked List

Definition:

A **linked list** is a linear data structure consisting of **nodes**, where each node contains **data and a pointer (address) to the next node** in the list. Memory is non-contiguous

Diagram:

```
[Data | Next] → [Data | Next] → [Data | NULL]
```

Example:

Dynamic student record management.

c) Stack

Definition:

A **stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle, where insertion and deletion are performed only from **one end called TOP**.

Operations:

- Push (Insertion)
- Pop (Deletion)

Diagram:

```
| 30 | ← Top
```

| 20 |
10

Example:

Undo operation in text editor.

d) Queue

Definition:

A **queue** is a linear data structure that follows the **FIFO (First In First Out)** principle, where insertion is done at the **REAR** end and deletion is done from the **FRONT** end.

Operations:

- Enqueue (Insertion)
- Dequeue (Deletion)

Diagram:

Front → [10] [20] [30] [40] ← Rear

Example:

Queue at ticket counter.

2. Non-Linear Data Structures

In non-linear data structures, data elements are **not arranged sequentially** and one element may be connected to **multiple elements**.

a) Tree

Definition:

A **tree** is a non-linear data structure that represents data in a **hierarchical form**, consisting of **nodes and edges**, with one root node.

Diagram:

A



Example:

File directory structure.

b) Graph

Definition:

A **graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges**, used to represent relationships between data elements.

Diagram:



Example:

Road map, social network.

Conclusion

Data structures help in efficient data storage, faster processing, and better memory utilization. Linear data structures like array, linked list, stack, and queue are easy to implement, while non-linear data structures like tree and graph handle complex relationships.

Define Best Case, Worst Case, and Average Case Time Complexity with Diagram

Time Complexity

Time complexity is a measure of the **time taken by an algorithm to run** as a function of the **input size (n)**. It helps us understand how efficient an algorithm is when input size increases.

Time complexity is generally expressed using **Big-O notation** such as $O(1)$, $O(n)$, $O(\log n)$.

1. Best Case Time Complexity

Definition

The **best case time complexity** is the **minimum time** required by an algorithm to execute when the **input is in the most favorable condition**.

Explanation

- Algorithm performs **minimum number of operations**
- Execution is **fastest**
- Happens very rarely in practical situations

Example (Linear Search)

If the element to be searched is found at the **first position**, only **one comparison** is needed.

Best Case Time Complexity:

$$O(1)$$

Diagram (Best Case – Linear Search)

Array: [5 | 10 | 15 | 20 | 25]

Search: 5

Comparison:

[5] ← Found in first step

2. Worst Case Time Complexity

Definition

The **worst case time complexity** is the **maximum time** required by an algorithm to execute when the **input is in the least favorable condition**.

Explanation

- Algorithm performs **maximum number of operations**
- Execution is **slowest**
- Gives the **upper limit** of running time
- Most important case for analysis

Example (Linear Search)

If the element is present at the **last position** or **not present**, all elements must be checked.

Worst Case Time Complexity:

$$O(n)$$

Diagram (Worst Case – Linear Search)

Array: [5 | 10 | 15 | 20 | 25]

Search: 25

Comparisons:

[5] → [10] → [15] → [20] → [25] ← Found last

3. Average Case Time Complexity

Definition

The **average case time complexity** is the **average time taken** by an algorithm to execute over **all possible inputs** of size n .

Explanation

- Calculated by considering **all positions**
- More realistic than best case
- Harder to calculate

Example (Linear Search)

On average, the element is found **somewhere in the middle** of the array.

Average Case Time Complexity:

$$O(n)$$

Diagram (Average Case – Linear Search)

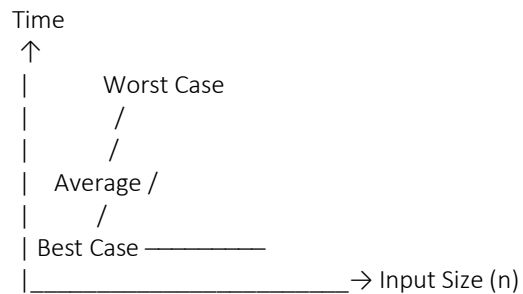
Array: [5 | 10 | 15 | 20 | 25]

Search: 15

Comparisons:

[5] → [10] → [15] ← Found in middle

Graphical Representation of Time Complexity



Comparison Table

Case	Meaning	Condition	Time Complexity
Best Case	Minimum time	Best input	$O(1)$
Average Case	Average time	Normal input	$O(n)$
Worst Case	Maximum time	Worst input	$O(n)$